

作业 2

——Max-Cut——

1. 题目

描述

一张图 (V, E) 的割 S 定义为边集 $\{(u, v) \in E : u \in S, v \notin S\}$ 。

请找一个割 S ，使得割的大小不少于总边数的 0.45 倍。

输入

输入数据第一行为数据组数 $T(T \leq 100)$ 。

接下来分别是 T 组数据。

每组数据第一行为正整数 n 和正整数 $m(n \leq 100, m \leq 2000)$ 。

之后 m 行，每行两个数 u 和 v ，为一条边的两个端点。可能有重边，但保证无自环。

输出

包含 T 组数据。

每组数据第一行为正整数 m ，表示 S 的大小。

第二行 m 个数，为 S 中点的编号。可按照任意顺序。

样例输入

```
1
4 4
1 2
2 3
3 4
4 1
```

样例输出

```
2
1 3
```

2. 思路

这个题目如果使用暴力枚举所有割的情况，时间复杂度是 $O(2^n)$ ，对于较大的 n 来说是不可行的。注意到我们输出的结果只需要是一个大于0.45倍总边数的割，因此我们可以使用随机算法来找到一个近似解。

假设所有的割是随机分布的，并且我们讲所有点随机地放到两个集合中，那么产生的割的数量的期望 $\mathbb{E} = 0.5m$ ，其中 m 是边数。期望值已经高于题目要求的0.45，因此，我们只需要多次试验，找到一个大于0.45 m 的割即可。但由于总的割的数目并不确定，我们无法判断找到的这个结果是否达到要求，因此，我们需要数学分析出：进行 N 次试验后，找到一个大于0.45 m 的割的概率。当这个概率足够大的时候，我们认为这件事是一定能办到的。

3. 算法实现

抛开数学不谈，我们假定试验次数为`const int TIMES = 300`，如果精度不够我们只需要调整这个数据取值即可。

3.1. 算法流程

由于这个题目要求进行 T 次输入，每次都几乎完全一样，因此，我们可以把操作放在一个 `while` 循环中：

```
int T;
cin >> T;
while(T--){
    // 每一轮都需要的操作
}
```

对于每一轮操作，我们先读取 n 和 m ，分别代表点的编号以及接下来总的边数；

然后输入每个边上的点对。我们可以使用 `pair<int, int>` 来储存每条边的两个端点，使用 `vector<pair<int, int>>` 来储存所有的边。

接下来，我们需要对每个点进行分组，我们不需要使用两个 `vector` 去记录每个组里面有什么元素，而是可以通过新建一个 `vector<int> flag` 来记录每个点所在的组别，0 代表第一组，1 代表第二组。

得到分组的结果之后，我们需要统计在这种分组结果之下能产生多少个割。我们只需要看看每个边的两个端点是否在同一个组里面，也就是它们的值是否相同，如果不同，那么这个边就会产生一个割，我们就把这个边的数量加 1。这里我新学到了亦或的用法，`[u] ^ flag[v]` 的结果如果是 1，说明它们在不同的组里面，如果是 0，说明它们在同一个组里面。

最后，我们来看看这个分组方法下是不是已有的所有分组方法里面最优的，如果是，我们需要记录产生的割的数目，以及用一个 `vector<int> ans` 来记录这个分组方法下的割的点的编号。

3.2. 代码实现

```
#include <bits/stdc++.h> //在实际写代码的时候，这不是一个好习惯，因为这会给编译器带来一些麻烦。
using namespace std;    //这同样不是个很好的习惯，更好的是写std::cout, std::cin等等。
const int TIMES = 300;
int main()
{
    int T;
    cin >> T;
    srand(time(0));
    while (T--)
    {
        int n, m; // n points, m inputs
        cin >> n >> m;
        vector<pair<int, int>> edge;
        for (int i = 1, v, u; i <= m; i++)
        {
            cin >> v >> u;
            edge.push_back({v, u});
        }
        vector<int> flag(n, 0), ans(n, 0);
        int maxn = 0; // 300 random attempts to find a good solution
        while (TIMES--)
        {
            for (int i = 0; i < n; i++)
                flag[i] = rand() % 2; // Randomly assign each point to group 0 or 1
            int sum = 0;
            for (int i = 0; i < m; i++)
            {
                if (flag[edge[i].first - 1] ^ flag[edge[i].second - 1])
                    sum++;
            }
            if (sum > maxn) // Refresh the best solution found so far
```

```
        {
            maxn = sum;
            for (int i = 0; i < n; i++)
                ans[i] = flag[i];
        }
    }
    int num = 0;
    for (int i = 0; i < n; i++)
        num += ans[i];
    cout << num << endl;
    for (int i = 0; i < n; i++)
        if (ans[i])
            cout << i + 1 << " ";
    cout << endl;
}
return 0;
}
```

3.3. 一点补充

这里的随机数其实是“伪随机”。事实上，要生成一个真随机数是必须要借助物理过程才能实现的，比如说放射性衰变、热噪声等等。我们这里使用的随机数生成器是基于算法的，虽然它们在统计上表现得像随机数，但它们是完全确定的，只要知道了初始状态（种子），你就可以预测出所有的随机数。因此，我们通常称它们为“伪随机数”。“伪随机”也有优劣之分，在这个课程当中，对随机性要求并不会很高，因此，一般的随机数生成函数几乎都可以满足要求。

作业 3

——Fingerprint——

4. 题目

描述

本题将会给你三个 $n \times n$ 的矩阵 A, B, C ，你需要判断 AB 是否等于 C 。

由于输入规模较大，我们在输入中只给出 C ，而 A, B 中的元素由以下函数生成：

```
int randint(const unsigned int &seed) {  
    static unsigned int x = seed;  
    x = (x * 23333LL + 23327) % 1000000007;  
    return x % 20 - 10;  
}
```

其中 $seed$ 将会在输入中给出。生成矩阵 A, B 的代码如下：

```
for(int i = 1; i <= n; i ++)  
    for(int j = 1; j <= n; j++) {  
        a[i][j] = randint(seed);  
        b[i][j] = randint(seed);  
    }
```

输入

测试数据第一行为一个正整数 $n \leq 3000$ ，表示矩阵的大小。第二行为一个不超过 10^9 的正整数，表示 $seed$ 。接下来 n 行，每行包含用空格分开的 n 个整数，表示矩阵 C 。

输出

输出 “YES” 或 “NO”（不包含引号），表示 $AB = C$ 是否成立。

样例输入

```
3  
462191211  
2 -67 106  
102 42 -2  
-63 -57 47
```

样例输出

NO

5. 思路

由于矩阵的大小 n 可以达到 3000，直接计算 AB 的时间复杂度为 $O(n^3)$ ，这在实际中是不可行的。我们可以使用随机化算法来验证 AB 是否等于 C 。

由数学知识，我们知道如果 $AB = C$ ，那么对于任意向量 x ，都有 $A(Bx) = Cx$ 。因此，我们可以随机生成一个向量 x ，计算 Bx 和 Cx ，然后再计算 $A(Bx)$ ，最后比较 $A(Bx)$ 和 Cx 是否相等。

在这个算法中，我们能够会得到：

- 1) 如果 $AB = C$ ，我们一定会输出 “YES”。
- 2) 如果 $AB \neq C$ ，我们可能输出 “YES”，只是这个概率应该足够低。

我们需要的数学知识：

$$\begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} a_{11} \cdot x_1 + a_{12} \cdot x_2 + \dots + a_{1n} \cdot x_n \\ a_{21} \cdot x_1 + a_{22} \cdot x_2 + \dots + a_{2n} \cdot x_n \\ \vdots \\ a_{n1} \cdot x_1 + a_{n2} \cdot x_2 + \dots + a_{nn} \cdot x_n \end{pmatrix}$$

因此，我们可以创建一个函数来进行 $n \times n$ 矩阵与 n 维向量乘法，这样可以简化主函数的代码。

5.1. 解题步骤

首先，我们需要按照要求生成矩阵 A, B ，然后输入 C 矩阵。

接着，我们来书写矩阵与向量的乘法函数：我们需要传入一个 $n \times n$ 的矩阵和一个 n 维的向量，输出一个 n 维的结果向量。通过之前的数学知识铺垫，我们容易通过循环求出这个向量的各个分量。

```
void MatrixMultiplyVector(const vector<vector<int>> &A, const vector<int> &x, vector<int> &res) {
    int n = A.size();
    for(int i = 0; i < n; i++) {
        res[i] = 0;
        for(int j = 0; j < n; j++) {
            res[i] += A[i][j] * x[j];
        }
    }
}
```

之后，我们需要生成一些随机地 n 维列向量，来进行检测。为了避免一些可能的巧合，我们使用我们自己的随机函数来生成这个向量。（我不确定用题目给的这个随机函数效果如何，也许也可以实现目标。）

最后我们只需要检查 $A(Bx)$ 和 Cx 是否相等，如果不相等，我们就输出“NO”，如果相等，我们继续进行下一轮的检测，直到完成所有的检测之后才输出“YES”。

还是老规矩，我们只有数学推导才能严格说明应该生成检测几个随机的 x ，因此在完成这个题目的时候，我们可以先设置一个常数 TIMES 来控制检测的次数，如果精度不够，我们只需要调整这个数据取值即可。

5.2. 代码实现

```
#include<bits/stdc++.h>
using namespace std;
const int TIMES = 10;
int randint(const unsigned int &seed) {
    static unsigned int x = seed;
    x = (x * 23333LL + 23327) % 1000000007;
    return x % 20 - 10;
}
void MatrixMultiplyVector(const vector<vector<int>> &A, const vector<int> &x, vector<int> &res) {
    int n = A.size();
    for(int i = 0; i < n; i++) {
        res[i] = 0;
        for(int j = 0; j < n; j++) {
            res[i] += A[i][j] * x[j];
        }
    }
}
int main(){
    int n;
    cin >> n;
    vector<vector<int>> a(n,vector<int>(n)),b(n,vector<int>(n)),c(n,vector<int>(n));
```

```

long long seed;
cin >> seed;
// Matrix C is input here
for(int i = 0; i < n; i++)
    for(int j = 0; j < n; j++)
        scanf("%d", &c[i][j]);    //scanf is faster than cin.

// Matrix A and B are generated here
for(int i = 0; i < n; i++){
    for(int j = 0; j < n; j++) {
        a[i][j] = randint(seed);
        b[i][j] = randint(seed);
    }
}
srand(time(0));    // To avoid some possible coincidences, we use our own random function.

for(int i = 0; i < TIMES; i++){
    // randomly generate a vector in n dimensions
    vector<int> x(n);
    for(int i = 0; i < n; i++)
        x[i] = rand() % 100000;
    vector<int> ResBX(n), ResCX(n), ResABX(n);
    MatrixMultiplyVector(b, x, ResBX);
    MatrixMultiplyVector(c, x, ResCX);
    MatrixMultiplyVector(a, ResBX, ResABX);
    if(ResABX != ResCX) {
        cout << "NO" << endl;
        return 0;
    }
    else
        continue;
}
cout << "YES" << endl;
return 0;
}

```

作业 4

——Rejection Sampling——

6. 题目

描述

给定实数 p ，要求用一个均匀0/1生成器，输出0/1 bit，使得输出1的概率为 p 。

以代码段填空的形式提交，你要写的代码是一个完整的代码中的一个函数 `bern`，传入 `double` 类型的实数 p 和均匀0/1 bit 随机生成器指针 `G`，返回一个0/1 bit。使用 `G->rand()`来获得均匀随机0/1 bit。你可以进入提交页面来查看你的代码提交形式的细节。

测试时，我们会调用你所写的函数 $10^5 \sim 10^6$ 次，如果频率在 $[p - \varepsilon, p + \varepsilon]$ 之间则视为通过， $\varepsilon = 0.01$ 。利用 `chernoff bound`，如果你的函数输出1的概率恰好为 p ，那么你有极高的正确率。

题目只有一次测试，总共调用你的函数不超过 2×10^7 次，时间限制5秒。

请注意，你只能使用传入的均匀0/1 bit 随机生成器来引入随机性，我们会检验你是否引入了额外的随机性。

```
#include <iostream>
#include <cassert>
#include <random>
using namespace std;
const int T = 1e6;
const double eps = 1e-2;
random_device rd;
class Generator{
private:
    minstd_rand gen;
public:
    int rand(){ return gen() % 2; }
};
int bern(double p, Generator *G){
// 在此处补充你的代码
// 你可以使用G->rand()来获得均匀随机0/1 bit
}
int main(){
    double p; int T;
    while (cin >> p >> T){
        Generator RAND, CHECK;
        int tmp = bern(p, &CHECK);
        for (int i = 1; i <= T; ++ i){
            Generator CHECK;
            if (tmp != bern(p, &CHECK)){
                // cout << "using external randomness!" << endl;
                cout << "WA" << endl;
                return 0;
            }
        }
        int c = 0;
        for (int i = 1; i <= T; ++ i)
            c += bern(p, &RAND);
        // cout << "the frequency is : " << (double)c / T << endl;
        if (c < (p - eps) * T || c > (p + eps) * T){
            cout << "WA" << endl;
        }
    }
}
```

```

        return 0;
    }
}
cout << "AC" << endl;
return 0;
}

```

输入

一个实数 $p \in (0, 1)$, $p \in \mathbb{R}$, 和一个正整数 n 表示调用次数。

输出

如果你的函数输出 1 的概率在 $[p - \varepsilon, p + \varepsilon]$ 之间, 并且没有引入额外的随机性, 则输出“AC”, 否则输出“WA”。

样例输入

0.5 1000000

样例输出

AC

提示

以下是对于调试的建议：建议直接拷贝提交页面的代码前后缀, 并在此基础上进行补全和调试。补全后的程序每次运行, 每行输入一个概率 p , 和一个运行次数 T , 会返回你的函数在 T 次运行之后的频率, 如果频率超出范围, 会返回 `assertion failed`。以上的输入输出格式以及样例, 是针对完整代码的。

7. 思路

题目要让我们干什么? 用一个硬币, 生成一个概率为 p 的随机数。这道题目中的 $G \rightarrow \text{rand}()$ 可以理解成就是一枚硬币。

这是一道数学题。让我们先回到高中的概率统计篇。首先, 我们复习一下条件概率:

$$p(AB) = p(A|B)p(B),$$

$p(AB)$ 代表 A 和 B 同时发生的概率, $p(B)$ 代表事件 B 发生的概率, $p(A|B)$ 代表在事件 B 发生的条件下, 事件 A 发生的概率。我们可以把这个公式理解为: 如果我们知道了事件 B 发生了, 那么事件 A 发生的概率就是 $p(A|B)$, 而事件 AB 同时发生的概率就是 $p(A|B)p(B)$ 。

接下来, 我们思考如果要生成 $p = \frac{1}{4}$ 的随机数, 有什么想法?

很容易想到, 我们可以抛两次硬币, 如果两次都是正面, 那么我们就输出 1, 否则输出 0。这样的话, 输出 1 的概率就是 $\frac{1}{4}$, 输出 0 的概率就是 $\frac{3}{4}$ 。类似的, 如果要生成 $p = \frac{1}{8}$ 的随机数, 我们可以抛三次硬币, 如果三次都是正面, 那么我们就输出 1, 否则输出 0。这样的话, 输出 1 的概率就是 $\frac{1}{8}$, 输出 0 的概率就是 $\frac{7}{8}$ 。

这些 p 很容易想到的原因在于, 这都正好是 $(\frac{1}{2})^k$, $k \in \mathbb{Z}$ 。但实际上的 $p \in \mathbb{R}$ 并不是那么好处理。我们继续尝试更一般的 p , 例如 $p = \frac{1}{3}$ 。我们可以抛两次硬币, 如果两次都是正面, 那么我们就输出 1, 如果两次是一正一反, 那么我们就输出 0, 否则我们就重新抛两次硬币, 并按照之前的规则输出, 直到产生输出为止。这样的话, 输出 1 的概率就是 $\frac{1}{3}$, 输出 0 的概率就是 $\frac{1}{3}$ 。这可以理解成 “Rejection Sampling”, 我们在这个过程中拒绝了两次都是反面的情况。

用条件概率来写就是: A 代表两次都是正面, B 代表两次不都是反面, 那么

$$p(A) = p(AB) = \frac{1}{4}; p(B) = \frac{3}{4}; p(A|B) = \frac{p(AB)}{p(B)} = \frac{1}{3}.$$

我们抛硬币的次数实际上是由将 p 转化成二进制小数的位数决定的。具体来说,

$$\frac{1}{3} = 0.0101010101\dots_2$$

因此，我们需要抛两次硬币来生成 p ，如果两次都是反面，那么我们就重新抛两次硬币，直到产生输出为止。

至此，我们的思路已经很清楚了：首先，我们需要将 p 表示成二进制小数的形式，也就是寻找 $p_1, p_2, \dots, p_n$ ，使得：

$$\sum_{i=1}^n p_i \cdot \left(\frac{1}{2}\right)^i = p$$

然后，我们抛硬币（调用随机 0/1 生成函数），检验生成的数字是否等于 p_1 ，如果等于，则输出；如果不等，则重新生成随机 0/1 数字，并与 p_2 比较，以此类推。

8. 代码实现

```
int bern(double p, Generator *G){
    if(p >= 1.0){
        return 1;
    }
    while(true){
        p *= 2;
        int dif = (int)(p);
        p -= dif;
        int r = G -> rand();
        if(r == dif)
            return r;
    }
}
```

9. 注释

这个题目用 Sampling Rejection 理解对我而言是一件不容易的事情，因为二进制表示与 $p = \frac{1}{3}$ 的例子的区别还是很大的。经过一个下午的捶打，我理解二进制小数表示之后，考虑这个循环小数的一个周期内，如果出现了随机数与 p_i 相同的情形，就相当于把之后的都接受了，直到如果第一个周期走完都没输出任何数，就是拒绝了这个周期的情况。

在思考的过程中，二进制的表示也可以有另一种解释方式。这种解释非常符合代码本身传递的逻辑。

对于一个实数 p ，我们假定它大于 $\frac{1}{2}$ ，那么，我们要先满足这 $\frac{1}{2}$ 的概率输出 1，也就是有至少一般的情形，我们应该输出 1，而不是输出 0。于是我们完成第一次操作：得到一个随机数，如果是 1 就输出，如果不是，也不输出 0，而是继续进行下一轮的操作。对于下一轮的操作，已经满足输出 1 的 $\frac{1}{2}$ 的概率了，那么之后就之应该有 $p - \frac{1}{2}$ 的概率输出 1 了。于是问题回到了原来的样子；如果 p 小于 $\frac{1}{2}$ ，我们就先满足输出 0 的 $\frac{1}{2}$ 的概率了，那么之后就之应该有 p 的概率输出 1 了。于是问题与之前的问题是一样的。

至于为什么是二进制下的小数，这是因为我们手上的硬币是均匀分布的。

作业 5

——亚线性时间估算分位点——

声明：题目来源于 <http://openjudge.cn/>，解题思路参考教授上课讲解内容，代码实现由本人完成。分享这个文档仅限于学术交流，不用于任何形式的商业用途。

10. 题目

描述

有 n 个互不相同的数 $a_1, \dots, a_n$ ，你只能通过给出的函数接口来访问某一编号 i 的数的大小 a_i ，并返回你估计的这 n 个数的 p 分位数， $0 \leq p \leq 1$ ，0 分位数表示最小值，1 分位数表示最大值。

具体地，设你输出的数在 n 个数中从小到大的排名为 r ，要求 $r \in [(p - e)n, (p + e)n]$, $e = 0.03$

以交互题的形式提交，你需要实现一个函数 `rnk (n, p)`，返回估计的 p 分位数，唯一访问 a 的方式是利用 `query (i)` 来得到 a_i ，代码需包含头文件 `rank.h`。

$10^5 \leq n \leq 10^6$ ，要求访问次数不超过 5000。

输入

第一行一个正整数 n 和一个实数 p 。

第二行 n 个 `int` 范围内的整数，表示这 n 个互不相同的数 a_1 到 a_n 。

注意：这里描述的是链接后的完整程序的输入/输出格式，仅供测试用途。你提交的程序只能通过 `query` 函数访问数据，请勿自行读入！

输出

无

样例输入

```
5 0.5
1 2 3 4 5
```

样例输出

```
your answer is : 3
the times you sample is : 5
the rank of your answer is : 3
correct!
```

提示

在以下链接下载交互库 `grader.cpp`，`rank.h`，以及样例代码 `rank_sample.cpp`：

<https://github.com/crasysky/rank/tree/main>

另外，本题只考虑 n 较大的情况，样例仅供参考输入输出格式。

11. 思路

这道题目已经把思路说得非常详细了。我们主要说说为什么需要这样做。传统上，我们可以对所有数据进行排序之后寻找 p 分位数，但是排序的时间复杂度是 $O(n \log n)$ 。这需要存储访问整个数据集。然而，如果这些数据分布式存储在非常多的机器硬盘上，无法载入内存，或者开销太大，我们能否找到一个近似解答呢？¹

根据题目的意思，对于所有数据，我们不能全部获取。我们只能选择性地访问其中的至多5000个数据。这类似于“抽样调查”，通过抽样的结果来获得总体的情况。我们可以随机地选择5000个数据进行访问，然后对这些数据进行排序，找到其中的 p 分位数。由于这些数据是随机抽取的，因此它们的分布应该与总体数据的分布相似，因此我们得到的 p 分位数应该也是一个近似解。

接下来，我们从数学上论证这样做法的精确程度：

为了简化问题，我们考虑 $p = 0.5$ 的情况，也就是中位数。我们希望输出的数据在 $(0.5 - 2\varepsilon, 0.5 + 2\varepsilon)$ 范围内。设 $\alpha = 0.5 - 2\varepsilon, \beta = 0.5 + 2\varepsilon$ ，考虑子集 A_1, A_2, A_3 分别代表排序后位次是 $[1, \alpha n], [\alpha n + 1, \beta n], [\beta n + 1, n]$ 的数。对于一个平均采样，各有 $0.5 - 2\varepsilon$ 的概率落在 A_1, A_3 中。

这里需要 Chernoff Bound 不等式：²

$$\Pr\left(|\bar{X} - \mu| \geq \sqrt{\frac{\log(\frac{2}{\delta})}{2n}}\right) \leq \delta$$

其中， $\bar{X}$ 是样本的均值， μ 是它的期望值， n 是采样的次数， δ 是允许的失败概率。

在这里，我们设定 $\delta = 2\varepsilon$ ，式子变成

$$\Pr\left(|\bar{X} - \mu| \geq \sqrt{\frac{\log(\frac{1}{\varepsilon})}{2n}}\right) \leq 2\varepsilon$$

根据题目， $\varepsilon = 0.015$ ，因此， $\sqrt{\frac{\log(\frac{1}{\varepsilon})}{2n}} < 0.015$ ，当 $n > 2334$ 时，这个不等式成立。因此，采样5000次是足够的。

12. 代码实现

```
#include <bits/stdc++.h>
#include "rank.h"
using namespace std;
int rnk(int n, double p){
    minstd_rand gen;
    int T = 5000;
    vector<int> v;
    for (int i = 1; i <= T; i++)
    {
        int idx = gen() % n + 1;
        v.push_back(query(idx));
    }
    sort(v.begin(), v.end());
    return v[max(int(p * T) - 1, 0)];
}
```

¹曾经我们做的题目都是要求输出一个精确的解答，但是在实际生活中，由于采样的数据可能都是含有很多不可避免的噪声的，因此，我们得到的所谓的精确的解答并不一定是真正精确的。与此同时，我们还花费了很多资源在求解这样的“理论上精确解”，这并不是很好的，因此，很多时候，我们只需要得到一个近似解，并且保证它的精度不会太差就可以了。

²这里的log是指以 e 为底的对数。并且要求 $X_i \in [0, 1]$ 。对于更大的数据，可以通过数学手段映射到 $[0, 1]$ 区间内。

作业 6

——d 维 1-median 查询——

声明：题目来源于 <http://openjudge.cn/>，解题思路参考教授上课讲解内容，代码实现由本人完成。
分享这个文档仅限于学术交流，不用于任何形式的商业用途。

13. 题目

描述

已知3维空间中 n 个点的坐标， q 次询问某一个点到这 n 个点的距离和。对于每次询问，设正确答案为 ANS ，要求输出的解在 $[(1 - e)ANS, (1 + e)ANS]$ 之内，其中 $e = 0.2$ 。

$n \leq 105, q \leq 105$, 坐标范围 $[-10^9, 10^9]$ 。

输入

第一行两个正整数，分别为 n, q 。之后 n 行，每行3个整数。每一行的3个整数描述一个点的坐标。之后 q 行，每行3个整数，每一行的3个整数描述一个需要查询的点的坐标。可能有重点。

输出

q 行，每行一个实数，表示查询的点到其他所有点距离和的估计。

样例输入

```
1 1
1 1 1
1 1 2
```

样例输出

```
1
```

提示

注意 long long 的范围

14. 分析

我们先做一些准备工作：

```
struct point{
    int x, y, z;
    point(){
        cin >> x >> y >> z;
    }
};
double dist(const point& a, const point& b){
    double dx = a.x - b.x;
    double dy = a.y - b.y;
    double dz = a.z - b.z;
    return sqrt(dx * dx + dy * dy + dz * dz);
}
```

这是对作业五的进一步分析讨论。在高中的《概率统计》一章中，我们学过分层随机抽样的概念，很显然，对于方差很大的一组数据，分层随机抽样是更合理的，更容易囊括奇特的数据并且控制它们对抽样的影响程度。对

于本题，我们可以将空间中的点按照距离中心点的远近分成若干层（比如10层），每层随机抽取一定数量的点来估计距离和。这样可以更好地控制误差，尤其是当存在一些点离“中心”很远的情况。

基于分层随机抽样的想法，我们按照以下步骤来实现：

首先，我们需要知道按照什么标准来分层。一个自然的选择是按照点到某个中心点（比如所有点的平均位置）的距离来分层。我们可以计算每个点到这个中心点的距离，然后根据距离将点分成若干层。于是，我们第一步就是要找到这个所谓的圆心。同样，我们不需要非常精确的圆心，所以策略是：任意选取一些点，然后在这些点里面找到距离其他所有点距离之和最小的点就可以。我们把这样的点视为一个可以接受的“圆心”。

```
int n, q;
cin >> n >> q;
vector<point> originalPoints(n), queryPoints(q);
// 选取中心点（距离和最小的点）
int K = min(n, 100);
srand(time(0));
vector<int> centerID(K);
for(int i = 0; i < K; ++i){
    centerID[i] = rand() % n;
}
double minDis = 1e18;
int FinalCenterID = 0;
for(int i = 0; i < K; ++i){
    double sum = 0;
    for(int j = 0; j < n; ++j){
        sum += dist(originalPoints[centerID[i]], originalPoints[j]);
    }
    if(sum < minDis){
        minDis = sum;
        FinalCenterID = centerID[i];
    }
}
```

接下来，我们计算每个点到这个中心点的距离，并按照距离将点分成若干层。我们可以根据距离的分布情况来动态地确定层数和每层的范围。比如，我们可以先计算所有点到中心点的距离，然后根据这些距离的分布情况来划分层次。这样可以更好地适应数据的实际分布，避免某些层过于稀疏或过于密集。

```
// 1. 计算每个点到中心点的距离
vector<double> dis(n);
double cen_sum = 0.0;
for(int i = 0; i < n; i++){
    dis[i] = dist(originalPoints[FinalCenterID], originalPoints[i]);
    cen_sum += dis[i];
}
// 2. 生成半径序列（从平均距离的0.1倍开始，每次翻倍，直到覆盖最远点）
const double eps = 0.1;
vector<double> r;
r.push_back(eps * cen_sum / n);
while (r.back() * 2 < *max_element(dis.begin(), dis.end())) {
    r.push_back(r.back() * 2);
}
// 确保最后一个半径能包含所有点
if (r.back() < *max_element(dis.begin(), dis.end())) {
    r.push_back(r.back() * 2);
}
// 3. 按距离排序索引
vector<int> idx(n);
for(int i = 0; i < n; i++) idx[i] = i;
```

```

sort(idx.begin(), idx.end(), [&](int i, int j) { return dis[i] < dis[j]; });

// 4. 动态分层（每一层对应一个半径区间）
vector<vector<int>> Layer;
int ptr = 0;
for (int i = 0; i < (int)r.size() && ptr < n; i++) {
    vector<int> tmp;
    while (ptr < n && dis[idx[ptr]] <= r[i] + 1e-9) {
        tmp.push_back(idx[ptr]);
        ptr++;
    }
    if (!tmp.empty()) Layer.push_back(tmp);
}
// 如果有点未被覆盖（理论上不会），加入最后一层
if (ptr < n) {
    vector<int> tmp;
    while (ptr < n) tmp.push_back(idx[ptr++]);
    Layer.push_back(tmp);
}

```

最后，我们按照距离将点分成若干层，并在每层中随机抽取一定数量的点来估计距离和。对于每个查询点，我们可以根据它到中心点的距离来确定它属于哪个层，然后在该层中进行随机抽样来估计距离和。

```

// 处理查询（每层最多采样300个点，按层大小加权）
for(int i = 0; i < q; i++){
    double ans = 0;
    for(auto& layer : Layer){
        int sz = layer.size();
        if(sz == 0) continue;
        double layer_sum = 0;
        int sample_cnt = min(300, sz);
        for(int j = 0; j < sample_cnt; j++){
            int idx = layer[rand() % sz];
            layer_sum += dist(queryPoints[i], originalPoints[idx]);
        }
        double layer_avg = layer_sum / sample_cnt;
        ans += layer_avg * sz;    // 加权：层平均距离 × 层大小
    }
    cout << ans << '\n';
}

```

15. 最后附上完整的代码

```

#include <bits/stdc++.h>
using namespace std;
using LL = long long;
struct point{
    int x, y, z;
    point(){
        cin >> x >> y >> z;
    }
};
double dist(const point& a, const point& b){
    double dx = a.x - b.x;
    double dy = a.y - b.y;
    double dz = a.z - b.z;
    return sqrt(dx * dx + dy * dy + dz * dz);
}
int main(){

```

```

int n, q;
cin >> n >> q;
vector<point> originalPoints(n), queryPoints(q);
// 选取中心点（距离和最小的点）
int K = min(n, 100);
srand(time(0));
vector<int> centerID(K);
for(int i = 0; i < K; ++i){
    centerID[i] = rand() % n;
}
double minDis = 1e18;
int FinalCenterID = 0;
for(int i = 0; i < K; ++i){
    double sum = 0;
    for(int j = 0; j < n; ++j){
        sum += dist(originalPoints[centerID[i]], originalPoints[j]);
    }
    if(sum < minDis){
        minDis = sum;
        FinalCenterID = centerID[i];
    }
}

// 1. 计算每个点到中心点的距离
vector<double> dis(n);
double cen_sum = 0.0;
for(int i = 0; i < n; i++){
    dis[i] = dist(originalPoints[FinalCenterID], originalPoints[i]);
    cen_sum += dis[i];
}

// 2. 生成半径序列（从平均距离的0.1倍开始，每次翻倍，直到覆盖最远点）
const double eps = 0.1;
vector<double> r;
r.push_back(eps * cen_sum / n);
while (r.back() * 2 < *max_element(dis.begin(), dis.end())) {
    r.push_back(r.back() * 2);
}
// 确保最后一个半径能包含所有点
if (r.back() < *max_element(dis.begin(), dis.end())) {
    r.push_back(r.back() * 2);
}

// 3. 按距离排序索引
vector<int> idx(n);
for(int i = 0; i < n; i++) idx[i] = i;
sort(idx.begin(), idx.end(), [&](int i, int j) { return dis[i] < dis[j]; });

// 4. 动态分层（每一层对应一个半径区间）
vector<vector<int>> Layer;
int ptr = 0;
for (int i = 0; i < (int)r.size() && ptr < n; i++) {
    vector<int> tmp;
    while (ptr < n && dis[idx[ptr]] <= r[i] + 1e-9) {
        tmp.push_back(idx[ptr]);
        ptr++;
    }
    if (!tmp.empty()) Layer.push_back(tmp);
}

```

```

    }
    // 如果有点未被覆盖（理论上不会），加入最后一层
    if (ptr < n) {
        vector<int> tmp;
        while (ptr < n) tmp.push_back(idx[ptr++]);
        Layer.push_back(tmp);
    }

    // 处理查询（每层最多采样300个点，按层大小加权）
    for(int i = 0; i < q; i++){
        double ans = 0;
        for(auto& layer : Layer){
            int sz = layer.size();
            if(sz == 0) continue;
            double layer_sum = 0;
            int sample_cnt = min(300, sz);
            for(int j = 0; j < sample_cnt; j++){
                int idx = layer[rand() % sz];
                layer_sum += dist(queryPoints[i], originalPoints[idx]);
            }
            double layer_avg = layer_sum / sample_cnt;
            ans += layer_avg * sz;    // 加权：层平均距离 × 层大小
        }
        cout << ans << '\n';
    }

    return 0;
}

```


作业 7

——k-HH 数据流算法——

声明：题目来源于 <http://openjudge.cn/>，解题思路参考教授上课讲解内容，代码实现由本人完成。

分享这个文档仅限于学术交流，不用于任何形式的商业用途。

16. 题目

描述

本题为交互题，交互格式与 OpenJudge - 1:估计 p 分位数相同，你的代码需要包含头文件“streaming.h”。

请实现一个数据结构，接受一个 $n \leq 10^6$ 个元素的输入数据流，并在最后输出数据流中出现次数较多的元素，具体要求如下：

你的程序输出中的每一个元素都在输入中出现了至少 $\frac{n}{k} - 0.01n$ 次。输入中每一个出现了至少 $\frac{n}{k}$ 次的元素都必须在输出中出现。你需要实现三个函数：

函数 `void init(int k)`。 k 的意义如前面所示， $k \leq 50$ 。

函数 `void add(int e)` 表示向你的数据结构插入一个新元素 e 。

函数 `std::vector report()`。我们会在输入数据流读入结束时调用该函数，请将满足题目要求的元素放到一个 `vector` 中并返回。

交互库中有一个长为 5000 的数组 `arr`，初始时全部为 0，你可以使用如下两个函数：

`void Set (int a, int b)`。表示将 `arr[a]` 设置为 b ，你需要保证 $0 \leq a < 5000$ 。

`int Get (int a)`。表示询问 `arr[a]` 的值，你需要保证 $0 \leq a < 5000$ 。

你不可以使用任何全局变量向 `report` 传递任何信息，否则本题将获得 0 分。

如下是一个错误的做法：在 `add` 时将 e 记录到自己声明的数组里，并在 `report` 中访问该数组。

输入

第一行两个正整数 n, k ，接下来一行 n 个正整数。

注意：这里描述的是链接后的完整程序的输入/输出格式，仅供测试用途。你提交的程序只能通过 `query` 函数访问数据，请勿自行读入！

输出

第一行一个整数 m 表示输出个数，接下来一行 m 个正整数。

注意：这里描述的是链接后的完整程序的输入/输出格式，仅供测试用途。你只需使用 `report` 返回答案即可。

样例输入

```
5 2
1 2 1 2 1
```

样例输出

```
1
1
```

提示

1) 在以下链接下载交互库 grader.cpp, streaming.h, 以及样例代码 example.cpp:

<https://github.com/crasysky/heavy-hitter/tree/main>

2) 你可以使用少量的局部数组（或 set, map）

17. 思路

题目描述比较抽象，我们一点一点来看。首先，题目明确要求，不能存储这些数据，也就是只能看一些数据，而不能获得总体情况。

我们先来介绍一个工具：哈希函数。我们可以使用哈希函数来将元素映射到一个较小的空间中，这样我们就可以在这个较小的空间中进行统计，而不需要存储所有的元素。具体来说，你可以理解成有很多个垃圾桶，我们对垃圾进行分类的时候，每个垃圾都会被扔进一个桶中。显然，某个垃圾桶中的垃圾数量是大于某一种应该扔进这个桶中的垃圾数量。比如，你手上有塑料瓶，纸杯，易拉罐，水果核。你可以按照金属与非金属来分类，那么塑料瓶，纸杯和水果核就会被扔进一个桶中，易拉罐就会被扔进另一个桶中。现在，你再以另一种垃圾分类的方式，比如厨余垃圾和其他，那么纸杯，塑料瓶和易拉罐就会被扔进一个桶中，水果核就会被扔进另一个桶中。我们的算法是：如果想要统计水果核的数目，我们就统计两种方式中，包含水果核的垃圾桶的垃圾总数最小的那个桶，作为输出。

事实上，我们肯定不会只有两个垃圾桶，也不会只有两种不同的分类方式。假设你有足够多的分类方式，以及足够多的垃圾桶，那么这个结果应该是不会太离谱的。这个算法叫做“count-min sketch”。

更严谨地说，对于每一个哈希函数，都有很多个哈希桶。每当我们看到一个元素，我们就把它放到每个哈希函数对应的哈希桶中。对于每个元素，我们可以通过每个哈希函数来找到它所在的哈希桶，然后统计这个哈希桶中的元素数量。由于桶中元素个数必定大于我们目标的元素数量，所以我们要寻找最小值。

18. 代码实现

这道题目的代码相对比较零散，是通过不同的函数实现一些功能构成的。

18.1. 准备工作

头文件，一些需要用到的常量，我们先声明出来，并且标注好可爱的备注，避免自己在将来指着这段代码骂：哪个人写的?! 看都看不懂。

```
#include <bits/stdc++.h> //这真的不是好习惯哈
#include "streaming.h"
using namespace std;

void init(int k);
void add(int e);
vector<int> report();

void Set(int a, int b); // arr[a] = b, 0 <= a <= 5000
int Get(int a);        // 返回 arr[a] 的值

const int M = 10;      // 哈希函数个数
int cnt[M][300];       // Count-Min Sketch
int total = 0;         // 已处理元素总数
int seed[M];           // 每个哈希函数的随机种子
int K;                 // 候选集大小（由 init 传入）
```

18.2. 哈希函数设计

我们需要设计一个哈希函数来将元素映射到一个较小的空间中，这也就是之前提到的垃圾分类的具体方式。我们需要保证这个哈希函数的随机性，以避免哈希冲突过多。

```
int hashFunc(int x, int idx) {
    int t = x ^ seed[idx];
    return ((t % 300) + 300) % 300; // 安全取模，避免负数
}
```

18.3. 全部代码

这道题目的代码如下，注释写的比较详细了。每个函数的作用在前文中已经有比较详细的描述，这里不再赘述啦~

```
#include <bits/stdc++.h>
#include "streaming.h"
using namespace std;

void init(int k);
void add(int e);
vector<int> report();
void Set(int a, int b); // arr[a] = b, 0 <= a <= 5000
int Get(int a); // 返回 arr[a] 的值
const int M = 10; // 哈希函数个数
int cnt[M][300]; // Count-Min Sketch
int total = 0; // 已处理元素总数
int seed[M]; // 每个哈希函数的随机种子
int K; // 候选集大小（由 init 传入）
int hashFunc(int x, int idx) {
    int t = x ^ seed[idx];
    // 安全取模，避免负数
    return ((t % 300) + 300) % 300;
}

void init(int k) {
    K = k;
    total = 0;
    srand(k); // 用 k 作为随机种子，保证可重复
    for (int i = 0; i < M; i++) {
        seed[i] = rand();
        for (int j = 0; j < 300; j++) {
            cnt[i][j] = 0;
        }
    }
    // 清除候选槽位（前 K 个位置）
    for (int i = 0; i < K; i++) {
        Set(i, 0);
    }
}

int count(int x) {
    int res = INT_MAX;
    for (int i = 0; i < M; i++) {
        res = min(res, cnt[i][hashFunc(x, i)]);
    }
    return res;
}

void insert(int x) {
    // 1. 如果已经在候选集中，直接返回
```

```

        for (int i = 0; i < K; i++) {
            if (Get(i) == x) return;
        }
        // 2. 尝试找到空槽位
        for (int i = 0; i < K; i++) {
            if (Get(i) == 0) {
                Set(i, x);
                return;
            }
        }
        // 3. 无空槽：替换当前估计频率最小的候选元素
        int minIdx = -1;
        int minFreq = INT_MAX;
        for (int i = 0; i < K; i++) {
            int freq = count(Get(i));
            if (freq < minFreq) {
                minFreq = freq;
                minIdx = i;
            }
        }
        // 如果当前元素比最小候选更频繁，则替换
        if (count(x) > minFreq) {
            Set(minIdx, x);
        }
    }
    void check() {
        for (int i = 0; i < K; i++) {
            if (Get(i) != 0) {
                int x = Get(i);
                if (count(x) < 1.0 * total / K) {
                    Set(i, 0);
                }
            }
        }
    }
}

void add(int e) {
    total++; // 总计数增加
    check(); // 清理旧候选（基于新阈值）
    for (int i = 0; i < M; i++) {
        cnt[i][hashFunc(e, i)]++; // 更新草图
    }
    if (count(e) >= 1.0 * total / K) {
        insert(e); // 条件插入
    }
}

vector<int> report() {
    check(); // 确保返回的都是当前频繁的元素
    vector<int> res;
    for (int i = 0; i < K; i++) {
        if (Get(i) != 0) {
            res.push_back(Get(i));
        }
    }
    return res;
}

```

作业 8

——MinHash 近似 Jaccard Similarity——

声明：题目来源于 <http://openjudge.cn/>，解题思路参考教授上课讲解内容，代码实现由本人完成。

分享这个文档仅限于学术交流，不用于任何形式的商业用途。

19. 题目

描述

给出 n 个单重集 $A_1, A_2, \dots, A_n$ ，每个集合都包含 m 个 $[0, 10^9]$ 中的整数。

有 q 次询问，每次询问给出 $i, j (1 \leq i, j \leq n)$ ，你的程序需要给出一个 $|A_i \cap A_j| / |A_i \cup A_j|$ 的近似解。

输入

第一行三个整数 $n, m, q (1 \leq n, m \leq 1000, 1 \leq q \leq 5 \times 10^5)$ 。接下来的 n 行，每行 m 个整数，表示集合 A_i 中的元素。接下来的 q 行，每行两个整数 i, j ，表示一组询问。

输出

对于每组询问，你需要输出一个浮点数，表示你给出的近似解。你需要保证，对于至少 99% 的询问：令你输出的解为 x ，标准答案为 y 。首先有 $x \geq 0$ ；其次要么有 $|x - y| < 0.15$ ，要么有 $\frac{\max(x, y)}{\min(x, y)} < 1.5$ 。

样例输入

```
3 2 2
1 2
2 3
1 3
1 2
1 3
```

样例输出

```
0.3333
0.3333
```

20. 分析

这个题目的意思非常简单：对于两个集合，我们定义他们的相似度是一个 $0, 1$ 之间的浮点数，表示它们交集与并集之比。我们需要给出这个比例的近似值。我们对精度的要求是，要么误差小于 0.15 ，要么相似度的比例不超过 1.5 倍。第二个限制条件是避免当真实值比较小 (< 0.3) 的时候，你随机输出都能正确。

算法思路是 MinHash 算法。具体来说操作步骤是

- 1) 找到很多个哈希函数，并将数据通过哈希函数映射到一个较小的空间中。
- 2) 我们找一个集合映射后最小的元素。
- 3) 对于相同的哈希函数，相同的元素经过映射后也应该得到相同的结果。因此，我们选择比较哈希函数中最小的数值是否相等。如果相等，那么我们就在原象中找到了两个相等的元素。
- 4) 通过计数，在足够多的不同的哈希函数中，统计最小值相等的次数占总哈希函数数量的比例，这个比例就是近似的 Jaccard 相似度。

于是，我们的思路已经很清楚了。接下来我们就来实现这个算法。

21. 算法实现

21.1. 我们需要很多个哈希函数

```
int hashFunction(int x, int t, vector<int>& k, vector<int>& mod){
    return (x ^ k[t]) * 1LL * k[t] % mod[t];
}
```

其中， k , mod 是预先生成的随机数。我们需要保证这些哈希函数的独立性，因此我们需要生成足够多的随机数。需要注意的是，这里的随机数不能是 0，否则除法会不合法。

```
srand(time(0));
// 生成T个随机哈希函数的参数
vector<int> k(T), mod(T);
for(int i = 0; i < T; i++){
    k[i] = rand() % 1000000000 + 1;
    mod[i] = rand() % 1000000000 + 1;
}
```

21.2. 对每个集合计算它的 MinHash

我们需要一个二维数组来存储每个集合在每个哈希函数下的最小哈希值。对于每个集合，我们遍历它的元素，计算它们在每个哈希函数下的哈希值，并更新最小值。

```
vector<vector<int>> minn(n, vector<int>(T, INT_MAX));
for(int i = 0; i < n; i++){
    for(int t = 0; t < T; t++){
        for(int j = 0; j < m; j++){
            int hashValue = hashFunction(a[i][j], t, k, mod);
            minn[i][t] = min(minn[i][t], hashValue);
        }
    }
}
```

21.3. 处理查询

对于每个查询，我们需要比较两个集合在每个哈希函数下的最小值是否相等，并统计相等的次数。最后我们输出相等次数占总哈希函数数量的比例。

这里需要小心 C++ 的整数除法问题，我们需要将相等次数转换为浮点数进行除法运算。

```
for(int i = 0; i < q; i++){
    int x, y;
    cin >> x >> y;
    x--; y--; // 转换为0-based索引
    double similarity = 0.0;
    for(int t = 0; t < T; t++){
        if(minn[x][t] == minn[y][t]){
            similarity++;
        }
    }
    cout << double(similarity * 1.0 / T) << endl;
}
```

22. 完整代码

```

#include<bits/stdc++.h>
using namespace std;
const int T = 70; // 进行70次哈希函数

int hashFunction(int x, int t, vector<int>& k, vector<int>& mod){
    return (x ^ k[t]) * 1LL * k[t] % mod[t];
}

int main(){
    int n, m, q;
    cin >> n >> m >> q;
    //a[i]表示第i行的元素
    vector<vector<int>>> a(n, vector<int>(m));
    for(int i = 0; i < n; i++){
        for(int j = 0; j < m; j++){
            cin >> a[i][j];
        }
    }

    srand(time(0));
    // 生成T个随机哈希函数的参数
    vector<int> k(T), mod(T);
    for(int i = 0; i < T; i++){
        k[i] = rand() % 1000000000 + 1;
        mod[i] = rand() % 1000000000 + 1;
    }

    //计算每个集合中的最小哈希值

    vector<vector<int>>> minn(n, vector<int>(T, INT_MAX));
    for(int i = 0; i < n; i++){
        for(int t = 0; t < T; t++){
            for(int j = 0; j < m; j++){
                int hashValue = hashFunction(a[i][j], t, k, mod);
                minn[i][t] = min(minn[i][t], hashValue);
            }
        }
    }

    // 处理查询
    for(int i = 0; i < q; i++){
        int x, y;
        cin >> x >> y;
        x--; y--; // 转换为0-based索引
        double similarity = 0.0;
        for(int t = 0; t < T; t++){
            if(minn[x][t] == minn[y][t]){
                similarity++;
            }
        }
        cout << double(similarity * 1.0 / T) << endl;
    }

    return 0;
}

```

作业 9

——汉明距离³最近邻查询——

声明：题目来源于 <http://openjudge.cn/>，解题思路参考教授上课讲解内容，代码实现由本人完成。

分享这个文档仅限于学术交流，不用于任何形式的商业用途。

23. 题目

描述

有一个包含 n 个长度为64的01串的集合 A 以及 q 个询问，每次询问会给出一个长度为64的01串 x ，你需要找到在 A 中， x 在汉明距离下的最近邻的 5-近似，即，假设在 A 中与 x 的汉明距离最短的是 y ，你需要找到 $z \in A$ ，使得 $\text{dist}(x, z) \leq 5 * \text{dist}(x, y)$ 。这里 $\text{dist}(x, y)$ 表示 x 与 y 的汉明距离。注意，可能存在多个 5-近似的最近邻，你只需要输出其中任意一个即可。

输入

第一行两个正整数 $n, q \leq 100000$ 。

接下来 n 行，第 i 行是一个不超过 2^{64} 的正整数 a_i ，它的二进制表示一个 01 串（若长度不足 64 位，则补前导零）。

接下来 q 行，第 i 行是一个不超过 2^{64} 的正整数 x_i ，表示一个询问。

输出

你需要输出 q 行，第 i 行是一个在 1 到 n 之间的正整数 t_i ，表示 a_{t_i} 是 x_i 在汉明距离下的最近邻的 5-近似。

样例输入

```
5 5
679
793
129
91
406
613
98
534
752
595
```

样例输出

```
1
4
5
1
4
```

³汉明距离是以理查德·卫斯里·汉明命名的概念，以 $d(x, y)$ 表示两个字 x, y 之间的汉明距离，指两个等长字符串对应位置不同字符的数量。

24. 思路

由于数据数目很大，我们不能一个一个去数，否则必然超时。这道题需要介绍的算法叫做 LSH (Locality-Sensitive Hashing)，它的核心思想是通过构造一些哈希函数，使得相似的输入在哈希空间中也相似，从而可以快速地找到近似最近邻。

具体来说，我们的操作是，将每个 64 位的 001 串进行重新排列，然后检查前面 k 位是否相同，如果相同则认为它们是近似的。假设我们检查了前 10 个数字，他们都相同，那么最多这两个数字串有 54 个数字是不对应相等的。而这只是一个重排之后的结论。我们多用一些重排的方法，就可以控制住近似的程度。

接下来，我们详细来说这个题目本身。首先，我们得到的输入是十进制的整数，我们需要自己把它转化成二进制下的 01 串。其次，我们得到的 query 也是整数，也需要进行这样的转换，因此，我们为了简化问题，重复使用代码，写出这个转换的过程：

```
void Trans(ull x, vector<int>& binaryString) {
    for(int i = 0; i < 64; i++){
        if(x != 0){
            binaryString[i] = x % 2;
            x /= 2;
        }
        else
            binaryString[i] = 0;
    }
}
```

于是，问题的关键就转化到了如何实现将一个串进行重排。我们可以通过一个随机数生成器来生成一个长度为 64 的随机排列，然后按照这个排列来重排每个串。具体来说，我们可以使用 C++ 中的 `std::shuffle` 函数来实现这个功能。`shuffle` 函数可以随机打乱一个范围内的元素，因此我们可以创建一个包含 0 到 63 的数组，然后使用 `shuffle` 来打乱它，得到一个随机的排列。再将原来第 i 位的数字放到新 `shuffle` 排列得到的位置上，就完成了重排。

```
ull reorder(const vector<int>& binaryString, const vector<int>& p) {
    ull result = 0;
    ull power_of_two = 1;

    for (int newpos = 0; newpos < 64; newpos++) {
        int original_position = p[newpos];
        if (binaryString[original_position] == 1) {
            result += power_of_two;
        }
        power_of_two *= 2;
    }

    return result;
}
```

我们的这个重排函数返回的是重排后的二进制数在 10 进制下的表示。

另外，我们显然还需要一个计算汉明距离的函数：

```
int HamDis(const vector<int>& a, const vector<int>& b) {
    int dis = 0;
    for(int i = 0; i < 64; i++){
        if(a[i] != b[i])
            dis++;
    }
    return dis;
}
```

准备工作就做完了，我们现在开始写 main 函数。

需要注意的是，重排的方式必须相同才能进行前缀序列的比较，否则完全没有意义。

```
int main(){
    int n, q;
    scanf("%d%d", &n, &q);
    vector<ull> a(n);
    vector<vector<int>> binaryString(n, vector<int>(64));
    for (int i = 0; i < n; i++) {
        scanf("%llu", &a[i]);
        Trans(a[i], binaryString[i]);
    }

    vector<ull> queries(q);
    for (int i = 0; i < q; i++) {
        scanf("%llu", &queries[i]);
    }
    vector<vector<int>> queryBinaryStrings(q, vector<int>(64));
    for (int i = 0; i < q; i++) {
        Trans(queries[i], queryBinaryStrings[i]);
    }

    const int N = 20;
    vector<vector<int>> perm(N, vector<int>(64));
    mt19937 rng(12345);
    for (int k = 0; k < N; k++) {
        for (int i = 0; i < 64; i++) {
            perm[k][i] = i;
        }
        shuffle(perm[k].begin(), perm[k].end(), rng);
    }
}
```

这里没有使用 rand() 函数，而是使用了随机性更强的 std::mt19937，并且设置了一个固定的种子 (12345)，以确保每次运行程序时生成的随机排列都是相同的，这样可以保证结果的一致性。

接下来处理每个询问：

```
const int M = 10;
vector<vector<pair<ull, int>>> tables(N);

for (int k = 0; k < N; k++) {
    tables[k].resize(n);
    for (int i = 0; i < n; i++) {
        ull reordered_val = reorder(binaryString[i], perm[k]);
        tables[k][i] = make_pair(reordered_val, i);
    }
    sort(tables[k].begin(), tables[k].end());
}

vector<int> candidates;
candidates.reserve(N * 2 * M);

for (int qi = 0; qi < q; qi++) {
    candidates.clear();

    for (int k = 0; k < N; k++) {
        ull reordered_query = reorder(queryBinaryStrings[qi], perm[k]);
        int pos = lower_bound(
```

```

        tables[k].begin(),
        tables[k].end(),
        make_pair(reordered_query, -1)
    ) - tables[k].begin();

    int start = max(0, pos - M);
    for (int j = start; j < pos; j++) {
        candidates.push_back(tables[k][j].second);
    }

    int end = min(n, pos + M);
    for (int j = pos; j < end; j++) {
        candidates.push_back(tables[k][j].second);
    }
}

int best_idx = candidates[0];
int best_dist = HamDis(binaryString[best_idx], queryBinaryStrings[qi]);
for (int i = 1; i < candidates.size(); i++) {
    int idx = candidates[i];
    int dist = HamDis(binaryString[idx], queryBinaryStrings[qi]);
    if (dist < best_dist) {
        best_dist = dist;
        best_idx = idx;
    }
}
printf("%d\n", best_idx + 1);
}
return 0;

```

`lower_bound`: C++ 标准库的二分查找函数，返回第一个大于等于目标值的元素的位置。

`make_pair(reordered_query, -1)`: 排序表中的每个元素是 `pair<ull, int>`，比较时先比第一个元素（重排后的值），再比第二个元素（原始下标）。我们用 `-1` 作为第二个元素，因为 `-1` 是最小的整数，这样可以确保：

- 1) 如果排序表中有元素的重排值等于 `reordered_query`，`lower_bound` 会返回第一个这样的元素；
- 2) 如果没有，会返回第一个比它大的元素的位置。

`tables[k].begin()`: 把迭代器转换成整数下标，方便后续操作。

至此，我们成功解答完这道题目，我的咖啡也喝得只剩一半了。

我会把这些题解全部上传到我的 GitHub 上，地址是：

https://github.com/James-Yao-xj/PKU_Program_Practice_HW

如果你也对算法和编程感兴趣，欢迎来我的 GitHub!